



UNIVERSIDAD DE DAGUPAN

SCHOOL OF INFORMATION TECHNOLOGY EDUCATION

ITP10 | EVENT-DRIVEN PROGRAMMING

PRELIM | LABORATORY EXAM

Name : ED ANDREW A. RENDON
Year, Course & blk : 31-ITE-02
Subject : EVENT-DRIVEN PROGRAMMING
Date : 09/15/2026

SCORE:

Prelim Laboratory Examination: Interactive Student Performance Dashboard

Objective:

At the end of the examination, students should be able to develop a complete event-driven JavaScript web application by applying variables and constants, operators, conditional statements, loops, functions, arrays, objects, array methods, object destructuring, JavaScript modules, and DOM/event handling. Students must organize the program into reusable modules and connect user actions to the required application behavior.

Overview:

Develop an Interactive Student Performance Dashboard. The application will display student records, search and filter students, compute grades, classify academic status, calculate class summaries, and respond to user actions. The solution must demonstrate the major concepts covered in Modules 1–3: event-driven programming, JavaScript fundamentals and control structures, and functions/arrays/objects/modular programming.

Required Structure

```
student-performance-dashboard/  
├── index.html  
├── css/  
│   └── style.css  
├── js/  
│   ├── students.js  
│   ├── gradeUtils.js  
│   ├── display.js  
│   └── main.js
```

Requirements

1. Student Data Module - students.js

Create and export a constant array named students. Use the exact six records below. Keep the property names and initial values unchanged so the application can be evaluated consistently.

```
export const students = [  
  { id: 1, name: "Andrea Cruz", block: "31-ITE-01", quiz: 88, lab: 92, exam: 85 },  
  { id: 2, name: "Brian Santos", block: "31-ITE-01", quiz: 74, lab: 80, exam: 77 },  
  { id: 3, name: "Carla Reyes", block: "31-ITE-02", quiz: 95, lab: 94, exam: 96 },  
  { id: 4, name: "Daniel Garcia", block: "31-ITE-02", quiz: 68, lab: 72, exam: 70 },  
  { id: 5, name: "Erika Mendoza", block: "31-ITE-03", quiz: 82, lab: 87, exam: 84 },  
  { id: 6, name: "Francis Lim", block: "31-ITE-03", quiz: 59, lab: 65, exam: 61 }  
];
```

Use the following grade weights in all computations: Quiz = 25%, Laboratory = 35%, Prelim Exam = 40%. Do not hard code the final grades shown by the interface; they must be calculated from each object's scores.

2. Grade Utility Module - gradeUtils.js

Create and export the following reusable functions using the exact names below:

- calculateFinalGrade(student) — Return the numeric weighted grade using Quiz 25%, Laboratory 35%, and Exam 40%.

- `getAcademicStatus(grade)` — Use conditional logic to return: "Excellent" for 90 and above; "Passed" for 75–89.99; "Needs Improvement" for 70–74.99; and "Failed" for below 70.
- `searchStudents(students, query)` — Return students whose name contains the query. The search must be case-insensitive.
- `filterStudentsByBlock(students, block)` — Return all students when block is "All"; otherwise return only students from the selected block.
- `filterStudentsByStatus(students, status)` — Return all students when status is "All"; otherwise return students whose computed academic status matches the selected status.
- `calculateClassAverage(students)` — Return the numeric average of the computed final grades of the supplied array. Return 0 for an empty array.
- `countPassingStudents(students)` — Return the number of students with a final grade of 75 or higher.
- `getTopStudent(students)` — Return the student object with the highest computed final grade. Return null for an empty array.

Required JavaScript concepts in this module:

- Use `const/let` appropriately. Do not use `var`.
- Use arithmetic and comparison operators correctly.
- Use `if / else if / else` in `getAcademicStatus()`.
- Use `filter()` for searching/filtering.
- Use `reduce()` for `calculateClassAverage()` or for another required aggregate calculation.
- Use object destructuring in at least one reusable function.
- Use at least one arrow function as a callback supplied to an array method.

3. Display Module - `display.js`

Create reusable functions responsible only for displaying data in the webpage. At minimum, export these exact functions:

- `displayStudents(students)` — Render one student card/row for each supplied student.
- `displaySummary(students)` — Update the class average, passing count, total displayed students, and top-student name.
- `displayMessage(message)` — Show a message in the required message area.

Inside `displayStudents(students)`:

- Use a loop or `forEach()` to render the result set.
- Use object destructuring to access `id`, `name`, `block`, `quiz`, `lab`, and `exam`.
- Each rendered record must have the class `student-card`.
- Each card must display the student name, block, three component scores, computed final grade (two decimal places), and academic status.
- When the result set is empty, display exactly: "No students found".

4. Main Module - `main.js`

`main.js` must connect the data, utility functions, display functions, and user interface. It must:

- Import students from `students.js`.
- Import the required processing functions from `gradeUtils.js`.
- Import the required display functions from `display.js`.
- Select the required HTML controls using the exact IDs listed in this examination.
- Register event listeners using `addEventListener()`. Do not use inline `onclick` attributes.
- Display all six student records and the initial summary immediately when the page loads.
- Apply the search text, block filter, and status filter together when the user clicks Apply Filters.
- Reset all controls and display all six records again when the user clicks Reset.
- Update the visible summary based on the currently displayed result set.
- Display an appropriate message when a filter returns no results.

5. Required User Interface Elements

You may design the page freely, but the following IDs/classes, option values, and visible text must be used exactly so the interface can be tested reliably.

Interface Element	Required ID / Class	Required Text / Value	Purpose
Application title	—	Student Performance Dashboard	Visible title
Search input	searchInput	placeholder="Enter student name"	Search by name
Block filter	blockFilter	All, 31-ITE-01, 31-ITE-02, 31-ITE-03	Filter by block
Status filter	statusFilter	All, Excellent, Passed, Needs Improvement, Failed	Filter by status
Apply button	applyBtn	Apply Filters	Runs search + filters
Reset button	resetBtn	Reset	Restores initial state
Student list	studentList	—	Container for records
Student card	.student-card	—	One per displayed student
Class average	classAverage	—	Current result average
Passing count	passingCount	—	Current passing students
Displayed count	displayedCount	—	Current number of records
Top student	topStudent	—	Highest grade in current result
Message area	messageArea	No students found	No-result feedback

Suggested HTML Hooks

```
<input id="searchInput" placeholder="Enter student name">
<select id="blockFilter">...</select>
<select id="statusFilter">...</select>
<button id="applyBtn">Apply Filters</button>
<button id="resetBtn">Reset</button>
<div id="studentList"></div>
<span id="classAverage"></span>
<span id="passingCount"></span>
<span id="displayedCount"></span>
<span id="topStudent"></span>
<div id="messageArea"></div>
<!-- Each rendered record: <article class="student-card">...</article> -->
```

6. Event-Driven Behavior

- Apply Filters button click: read all three controls, process the data, display matching records, and update the summary.
- Reset button click: clear the search input, set both filters to All, remove any no-result message, and restore the initial records/summary.
- Search input keyup or input event: update the visible records as the user types. The Apply Filters button must still work even if you also implement live search.
- Block and status change events: you may update immediately, but your logic must still work correctly when Apply Filters is clicked.
- All interactions must be handled using `addEventListener()`.

7. Required Control-Structure Requirement

In addition to the condition used for academic status, create a function named `getPerformanceRemark(grade)` in `gradeUtils.js`. Use a switch statement or an equivalent `switch(true)` structure to return one of the following exact values:

- 90 or above: Outstanding
- 85–89.99: Very Good
- 80–84.99: Good
- 75–79.99: Satisfactory

- Below 75: Unsatisfactory
-

Display the performance remark for every student card. This requirement is included to assess decision-making/control structures from Module 2.

8. Module Configuration

Connect the main JavaScript file using the following exact module configuration:

```
<script type="module" src="js/main.js"></script>
```

Open the application through a local development server. ES modules may not work correctly when index.html is opened directly as a file.

Expected Initial Results

When the application first loads, all six students must be displayed. Based on the required dataset and grade weights, the application should produce the following initial summary. Values must come from your functions, not from hard-coded display text.

- Total displayed students: 6
- Class Average: 79.53
- Passing Students: 4
- Top Student: Carla Reyes (95.05)

Testing Requirements

Before submission, verify at least the following tests:

1. Displays all six students on initial page load.
2. Computes Andrea Cruz's final grade correctly using the required weights.
3. Classifies Carla Reyes as Excellent.
4. Classifies Francis Lim as Failed.
5. Searching for "carla" displays Carla Reyes even when lowercase is used.
6. Searching for a nonexistent name shows exactly "No students found".
7. Filtering block 31-ITE-02 displays only Carla Reyes and Daniel Garcia.
8. Filtering status Failed displays the correct failing record(s).
9. Combining a search term with block/status filters returns only records meeting all active conditions.
10. Reset restores the search box, filters, all six records, and initial summary.
11. Class average changes correctly when a filtered subset is displayed.
12. Top student changes correctly based on the currently displayed subset.
13. Every visible student record includes final grade, academic status, and performance remark.
14. No console errors appear during initial load, search, filtering, or reset.
15. All JavaScript imports/exports work through a local development server.

Automated-Grading / Validation Compatibility

- Do not rename the required files, exported functions, IDs, class names, option values, or exact no-results message.
- Do not hard-code individual final grades, status labels, summary values, search results, or filter results.
- Utility functions must work when the grader supplies different valid arrays/objects.
- Calculation functions must return numeric values. Format decimals only when displaying them.
- Students may choose any clean CSS design provided the required selectors and behaviors remain intact.
- The grader may check file existence, source structure, function return values, and browser behavior.

Submission Requirements

- Submit the complete student-performance-dashboard project folder/repository.
- All required files must be included and named correctly.
- The application must run without console errors using a local development server.
- Include one screenshot showing the initial dashboard and one screenshot showing a filtered result.

- The submitted work must be the student’s own implementation.

Screenshot 1:

Student Performance Dashboard

All

All

Apply Filters

Reset

Displayed Students: 6

Class Average: 79.53

Passing Count: 4

Top Student: Carla Reyes (95.05)

Andrea Cruz

ID: 1 | Block: 31-ITE-01

Quiz: 88 | Lab: 92 | Exam: 85

Final Grade: 88.20

Status: Passed

Remark: Very Good

Brian Santos

ID: 2 | Block: 31-ITE-01

Quiz: 74 | Lab: 80 | Exam: 77

Final Grade: 77.30

Status: Passed

Remark: Satisfactory

Carla Reyes

ID: 3 | Block: 31-ITE-02

Quiz: 95 | Lab: 94 | Exam: 96

Final Grade: 95.05

Status: Excellent

Remark: Outstanding

Daniel Garcia

ID: 4 | Block: 31-ITE-02

Quiz: 68 | Lab: 72 | Exam: 70

Final Grade: 70.20

Status: Needs Improvement

Remark: Unsatisfactory

Erika Mendoza

ID: 5 | Block: 31-ITE-03

Quiz: 82 | Lab: 87 | Exam: 84

Final Grade: 84.55

Status: Passed

Remark: Good

Francis Lim

ID: 6 | Block: 31-ITE-03

Quiz: 59 | Lab: 65 | Exam: 61

Final Grade: 61.90

Status: Failed

Remark: Unsatisfactory

Screenshot 2:

Student Performance Dashboard

31-ITE-01

Passed

Apply Filters

Reset

Displayed Students: 2

Class Average: 82.75

Passing Count: 2

Top Student: Andrea Cruz (88.20)

Andrea Cruz

ID: 1 | Block: 31-ITE-01

Quiz: 88 | Lab: 92 | Exam: 85

Final Grade: 88.20

Status: Passed

Remark: Very Good

Brian Santos

ID: 2 | Block: 31-ITE-01

Quiz: 74 | Lab: 80 | Exam: 77

Final Grade: 77.30

Status: Passed

Remark: Satisfactory

Grading Rubric

Criterion	Excellent	Proficient	Developing	Needs Improvement
Application Functionality (30 points)	All required behaviors work correctly, including initial display, combined search/filter, reset, summaries, status and remarks. (27–30)	Most required behaviors work; only minor defects are present. (22–26)	Several features work, but important behaviors are incomplete or incorrect. (16–21)	Application is substantially incomplete or does not function reliably. (0–15)

Semester: 1st

Academic Year: 2026-2027

Criterion	Excellent	Proficient	Developing	Needs Improvement
JavaScript Fundamentals & Control Structures (20 points)	Correct use of variables/constants, operators, conditionals, switch/control logic, loops/events, and correct grade/status computations. (18–20)	Most concepts are correct with minor logic issues. (14–17)	Some concepts are demonstrated but with noticeable errors. (10–13)	Required fundamentals/control structures are mostly missing or incorrect. (0–9)
Functions, Arrays & Objects (20 points)	Reusable functions, arrays of objects, array methods, callbacks/arrow function, reduce/filter, and destructuring are correctly applied. (18–20)	Most required concepts are used correctly. (14–17)	Concepts are partially used but several requirements are missing or incorrect. (10–13)	Little evidence of reusable functions/array/object processing. (0–9)
Event-Driven & Modular Organization (15 points)	Correct addEventListener usage; data, utilities, display, and controller responsibilities are separated with working imports/exports. (14–15)	Modules/events work with minor organizational issues. (11–13)	Partial modular/event-driven design; responsibilities or connections are unclear. (7–10)	Code is mostly placed in one file, uses incorrect event handling, or modules do not work. (0–6)
User Interface & Output (10 points)	Interface is clear, readable, complete, and all required IDs/classes/output fields are present. (9–10)	Usable interface with minor presentation/output issues. (7–8)	Interface or displayed information is incomplete. (5–6)	Interface is difficult to use or required outputs are largely missing. (0–4)
Code Quality, Testing & Submission (5 points)	Readable/indented code, meaningful names, no unnecessary repetition, no console errors, and complete submission. (5)	Minor code-quality or submission issue. (4)	Several quality/testing/submission issues. (2–3)	Submission is substantially incomplete or not testable. (0–1)

PREPARED BY:

ROCHELLE S. LANTO, MIT & JELSON V. LANTO, MIT
SITE, FACULTY